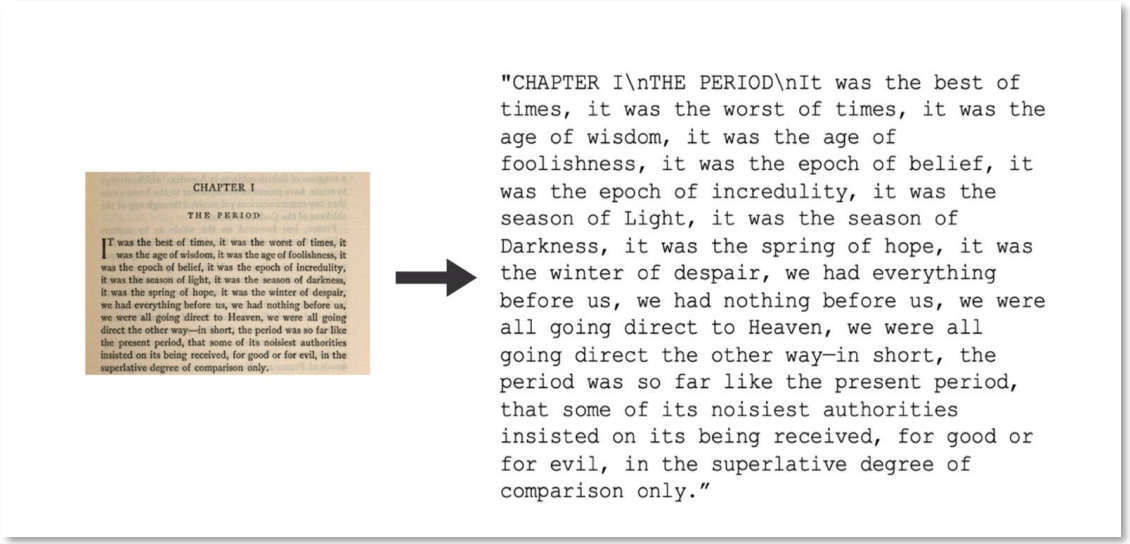
Natural Language Processing (NLP)

Chapter # 1: Traditional Text Processing Techniques

Sanjayae@bu.edu

NLP (Natural Language Processing)

Background

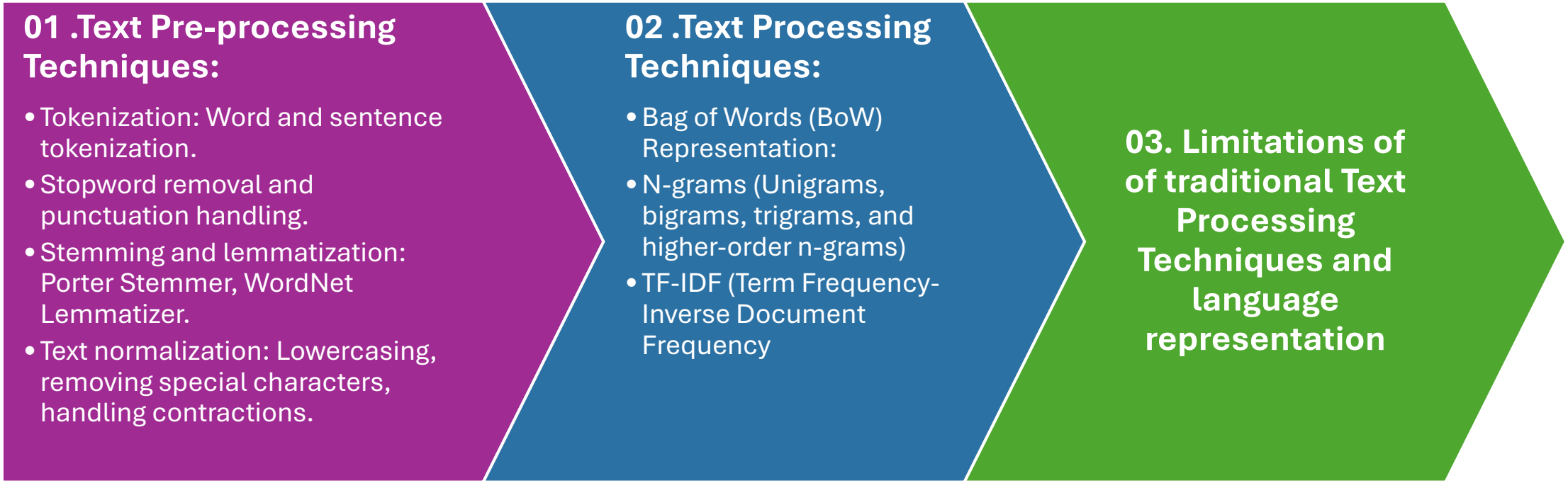


- Most basic version of Machine Language
- Foundation for all latest language models (LLMs) – Chat GPT, Grok , Gemini

Usage :

- Text Classification - News Classification, Spam Detection
- Semantic Analysis - Language understanding
- Content Generation – Create new content

Chapter #1 Scope



Text Pre-processing Techniques:

Text Pre-processing Techniques:

1. Tokenization: Word and sentence tokenization.
2. Stop-word removal and punctuation handling.
3. Stemming and lemmatization: Porter Stemmer, WordNet Lemmatizer.
4. Text normalization: Lowercasing, removing special characters, handling contractions.

Text Pre-processing Techniques ctd..

1. **Tokenization:**

Word Tokenization: This process splits text into individual words. For example, " Sri Lanka is a beautiful country in the world and University of Ruhuna is one of the beautiful places in the country . “ becomes,

"[Sri', 'Lanka', 'is', 'a', 'beautiful', 'country', 'in', 'the', 'world', ',', 'and', 'University', 'of', 'Ruhuna', 'is', 'one', 'of', 'the', 'beautiful', 'places', 'in', 'the', 'country', '.']

Sentence Tokenization: This involves dividing text into sentences. For example, "AI is advancing rapidly. It is reshaping industries." becomes ['AI is advancing rapidly.', 'It is reshaping industries.']. This will be useful for activities summarization or contextual analysis, where sentence-level structure matters.

Text Pre-processing Techniques ctd..

2. Stop-word Removal , Punctuation Handling:

Stop-words: Stop-words are common words like "is," "the," and "and" that do not carry significant meaning. Removing them reduces noise and helps algorithms focus on the informative parts of the text.

E.g. "The cat is on the mat“ >> "cat mat."

Punctuation Handling: Unnecessary punctuation marks, such as commas, periods, and special characters, are removed to simplify processing.

E.g. "Hello, world!" >> ”Hello world "

Text Pre-processing Techniques ctd..

3. Stemming and Lemmatization:

Stemming: Stemming reduces words to their root forms, often by chopping off affixes.

E.g. running, "runner," and "ran" >>> Run , (This can introduce non-dictionary word with no meaning)

Lemmatization: Lemmatization, on the other hand, reduces words to their base forms (lemmas) with context-awareness, producing linguistically accurate results. However, this method is more computationally more expensive.

E.g.

- Studies > Study
- Better > Good

Text Pre-processing Techniques ctd..

4. Text Normalization:

Lowercasing: Converts all text to lowercase (e.g., "AI" becomes "ai") to eliminate case sensitivity.

Removing Special Characters: Cleans text by eliminating characters like "@", "#", and "*" that are not essential for text analysis.

Handling Contractions: Expands contractions like "can't" to "cannot," ensuring uniformity and improving interpretability. This step is vital for downstream tasks like language modeling and translation

2. Text Processing Techniques

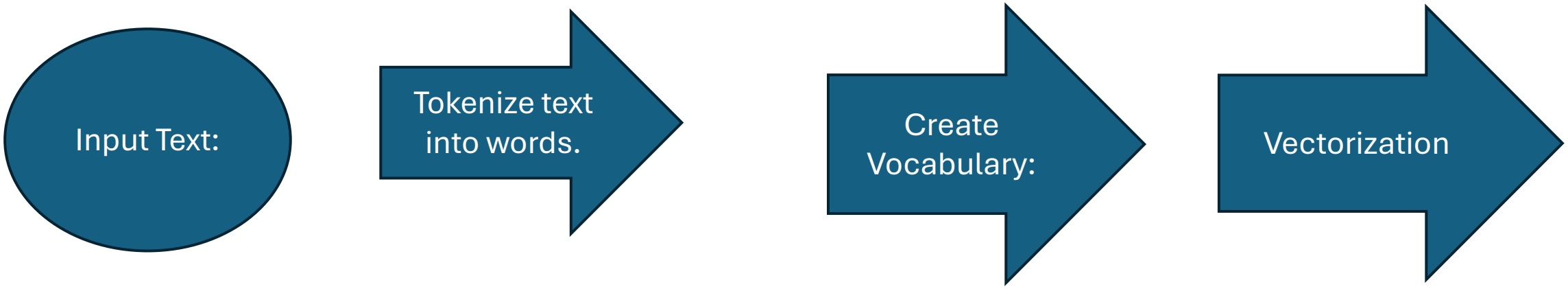
1. Bag of Words (BoW) Representation:
2. N-grams (Unigrams, bigrams, trigrams, and higher-order n-grams)
3. TF-IDF (Term Frequency-Inverse Document Frequency)

Bag of Words (BoW) in NLP

Bag of Words (BoW) is one of the simplest and most widely used techniques for text representation in Natural Language Processing (NLP). It converts text into a numerical representation that machine learning models can process.

- **Bag-like Representation**: :Treats text as a "bag" of individual words, disregarding grammar, word order, and punctuation.
- **Word Frequency**: The importance of a word is represented by its frequency (how many times it appears in the document).
- **Vocabulary:** BoW creates a vocabulary of all unique words in the dataset.
- **Vectorization :** Each document is represented as a vector and each dimension corresponds to a unique word in the vocabulary.
- The value of each dimension is the count (or sometimes binary presence) of the corresponding word in the document

Steps to Create a BoW Representation



Normalize text (e.g., lowercasing, removing special characters, handling stop words).

Identify all unique words in the dataset

Count the occurrences of each word in the vocabulary for each document.

Example:

Doc 1: "I love Galle."
Doc 2: “Galle is Beautiful."

Lowercase:
"i love galle", galle is beautiful.
Tokenize: ['i', 'love', ‘galle'], [‘galle', ‘beautiful']

['i', 'love', ‘galle', ' ‘beautiful']

Doc 1: [1, 1, 1, 0]
Doc 2: [0, 0, 1, 1]

Advantages and limitations of BoW Rep.

Advantages

1.Simplicity:

2. Efficiency: Works well for smaller datasets and simple tasks.

Limitations

1.High Dimensionality: For large datasets with a big vocabulary, vectors become very large

2.No Semantic Understanding, ignores the meaning or context of words. (E.g. Love vs like)

3.No Order Preservation: Loses information about the sequence of words. (E.g. Cat eats rat , Rat eats cat)

N-grams

N-grams are continuous sequences of n items (typically words or characters) from a given text or speech corpus used to model sequences of words to capture context and relationships between them.

Types of N-grams:

Unigram: Single word (n=1) E.g. "I love programming" → ['I', 'love', 'programming']

Bigram: Sequence of 2 words (n=2) E.g. "I love programming" → ['I love', 'love programming']

Trigram: Sequence of 3 words (n=3) E.g. "I love programming" → ['I love programming']

Higher-order N-grams: Sequence of n words (n > 3)

N-grams , applications and so on ...

Applications

Text Prediction: Use N-grams to predict the next word in a sequence.

E.g. In bigrams, "I love" → likely follow with "you" or "programming."

Text Similarity: Compare N-gram overlap to measure similarity between documents.

Machine Translation: Use N-grams to analyze word sequences for context-sensitive translations.

Sentiment Analysis: Identify common word patterns indicating positive or negative sentiment

Advantages:

Captures local context in text (e.g., phrases or word dependencies).

Simple and computationally efficient for many NLP tasks.

Limitations:

Data sparsity: Higher-order N-grams (e.g., trigrams) require more data to accurately estimate probabilities.

Context limitations: Does not account for long-range dependencies or sentence meaning.

TF-IDF (Term Frequency-Inverse Document Frequency)

- TF-IDF is a statistical method used in (NLP)
- Used for information retrieval to evaluate how important a word is to a document relative to a collection of documents (corpus).
- It is widely used in tasks like text mining, document classification, and search engines.

$$\text{TF-IDF}_{i,j} = \text{TF}_{i,j} \times \text{IDF}_i$$

$$\text{TF}_{i,j} = \frac{\text{Number of times term } i \text{ appears in document } j}{\text{Total number of terms in document } j}$$

- Measures how often a word appears in a document
- Higher frequency = Higher importance within the document.

$$\text{IDF}_i = \log \frac{\text{Total number of documents}}{\text{Number of documents containing term } i}$$

- Measures how unique or rare a word is across all documents in the corpus
- Words that appear in many documents (e.g., "the", "and") are less significant

TF-IDF (Term Frequency-Inverse Document Frequency) ctd..

Example Calculation

Corpus:

- Document 1: "I love programming."
- Document 2: "Programming is fun."
- Document 3: "I love fun activities."

Steps:

Vocabulary: (Unique words) : ['i', 'love', 'programming', 'is', 'fun', 'activities']

1. **TF (Term Frequency):**
   - Document 1: ['i': 1/3, 'love': 1/3, 'programming': 1/3]
   - Document 2: ['programming': 1/3, 'is': 1/3, 'fun': 1/3]
   - Document 3: ['i': 1/4, 'love': 1/4, 'fun': 1/4, 'activities': 1/4]
2. **IDF (Inverse Document Frequency):**
   - IDF(i) = log(Total Documents / Documents containing "i")
   - For i: log(3/2)=0.18
   - For “Love” : Log(3/2) = 0.18
   - For “programming “: log(3/2)=0.18
   - For “activities”: log(3/1)=0.48

TF-IDF for Document 1 :

TF-IDF: TF by IDF for each word in each document.

Document 1:

['i': 1/3 * 0.18 , 'love': 1/3 * 0.18 'programming': 1/3 * 0.18]

[0.62 , 0.62, 0.0.62]

Document 3 :

[i ; 1/3*0.18, ‘love’ : 1/3 * 1.8 , ‘fun’ : 1/4 * 0.48 , ‘Activities’ : 1/4 * 0.48]

[0.62 , 0.62, 0.12 , 0.12]

TF-IDF , ctd..

Advantages

Feature Weighting: Assigns higher weights to terms that are relevant to a document but not too common across the corpus.

Efficient Text Representation: Converts unstructured text into structured numeric data for machine learning models.

Simple and Effective: Works well for many NLP tasks as a baseline.

Limitations

Ignores Word Context: TF-IDF doesn't capture semantic meaning or word order. (E.g Good and “not good” treating independently)

Sparse Vectors: For large corpora, TF-IDF vectors become high-dimensional and sparse. (Computationally expensive)

Rare Word Bias: Words that are too rare may get overly high importance.

Applications

Search Engines: TF-IDF ranks documents based on query relevance.

Text Classification: Used to extract features for machine learning models.

Clustering and Topic Modeling: Helps identify key terms in documents.

3. Limitations of Traditional Representations

1. Lack of Semantic Understanding (E.g. Happy , Joyful treats completely differently)

2. High Dimensionality, for large corpora, this results in extremely sparse and computationally expensive matrices.

3. No Context Awareness - "I love dogs, not cats" and "I love cats, not dogs" produce similar BoW/TF-IDF vectors but have opposite meanings.

4. Difficulty with Polysemy and Synonymy

Polysemy**:** A single word with multiple meanings is treated uniformly. E.g. : "bank" (riverbank vs. financial institution) is represented identically.

Synonymy**:** Different words with similar meanings are treated separately. Example: "car" and "automobile" are treated as unrelated.

5. Poor Generalization :

Traditional representations are heavily reliant on the training corpus.

They fail to generalize well to unseen words or phrases (e.g., words not in the vocabulary).

6. No Handling of Long-Range Dependencies

Traditional methods can capture only local patterns (e.g., adjacent words in N-grams)

They fail to understand dependencies between words across a sentence or paragraph.

7. Fixed Vocabulary: Adding new documents or terms requires re-training to rebuild the vocabulary

8. Bias Toward Frequent Words: Common words (even if irrelevant) dominate feature vectors, particularly in BoW and TF-IDF.